# Uniwersytet Łódzki
# Wydział Matematyki i Informatyki

Temat ćwiczenia:

***Konwolucyjna sieć neuronowa***

Sprawozdanie z przedmiotu: ***Zaawansowane Metody Eksploracji***
Studia stacjonarne 2-go stopnia
Kierunek: *Analiza Danych, sem. II*
Prowadzący: *prof. dr hab. inż. Grzegorz Dudek*

| | |
|---|---|
| Data wykonania ćwiczenia: | 28.05.2025 |
| Data złożenia sprawozdania: | 04.06.2025 |
| Data zaliczenia: | |

| Wykonali: | Ocena: |
|---|---|
| **Dawid Sporysiak** | |

Nr sekcji: **3.19**

## A. Cel ćwiczenia

Celem ćwiczenia jest zbudowanie konwolucyjnej sieci neuronowej, wytrenowanie jej na dużym zbiorze cyfr z bazy, a następnie przetestowanie jej klasyfikacji na własnoręcznie napisanych cyfrach.

### Opis stosowanej sieci:

### Najważniejsze elementy:

Warstwa konwolucyjna Conv2D  wykrywa lokalne cechy obrazu za pomocą filtrów (jąder konwolucyjnych)

Warstwa poolingowa MaxPooling2D, która redukuje wymiarowość cech, zachowując najważniejsze informacje, wybiera maksymalną wartość w oknie 2x2.

Warstwa spłaszczająca Flatten służąca do przekształcania wielowymiarowych danych na wektor, aby móc je przekazać do warstw gęstych.

Warstwy gęste (Dense)

Hiperparametry zastosowane w danej sieci:
- Liczba filtrów w warstwie konwolucyjnej (16, 32, 64).
- Liczba neuronów w warstwie gęstej (32, 64, 128).
- Optymalizator: Adam do learning rate.
- Funkcja straty: sparse_categorical_crossentropy.

# Budowa modelu (Sequential):

```
model = tf.keras.Sequential([
   tf.keras.layers.Conv2D(params['filters'], (3, 3), activation='relu', input_shape=(28, 28, 1)),
   tf.keras.layers.MaxPooling2D((2, 2)),
   tf.keras.layers.Flatten(),
   tf.keras.layers.Dense(params['dense_units'], activation='relu'),
   tf.keras.layers.Dense(10, activation='softmax')
])
```

Conv2D: Warstwa konwolucyjna składająca się z:

- Params['filters'], Liczba filtrów
- (3, 3), Rozmiar jądra (okno 3x3 piksele).
- Activation='relu', Funkcja aktywacji ReLU

- Input_shape=(28, 28, 1), kształt obrazu, 28x28 pikseli z 1 kanałem
- MaxPooling2D((2, 2)), Redukuje wymiarowość o połowę (wybiera maksymalną wartość w oknie 2x2).
- Flatten(), Spłaszcza dane do 1D
- Params['dense_units'], Liczba neuronów
- Dense(10, activation='softmax'), 10 neuronów wyjściowych (10 cyfr) z funkcją aktywacji softmax

# Budowa model.compile():

```
model.compile(
   optimizer='adam',
   loss='sparse_categorical_crossentropy',
   metrics=['accuracy']
)
```

- optimizer='adam': optymalizator learning rate
- loss='sparse_categorical_crossentropy': Funkcja straty dla klasyfikacji
- metrics=['accuracy']: metryka (dokładność)

## B. Treść zadania

## Zadanie 1

## Treść:

1. Zaimportuj niezbędne moduły i ustaw ziarno generatora liczb pseudolosowych: `tf.random.set_seed(nr_gr)`, gdzie `nr_gr` to numer Twojej sekcji.

## Rozwiązanie zadania:

```
#Zadanie 1
import numpy as np
import matplotlib.pyplot as plt
import tensorflow as tf
from PIL import Image
from tensorflow.keras.datasets import mnist

nr_gr=3.19
tf.random.set_seed(nr_gr)
```

## Zadanie 2

## Treść:

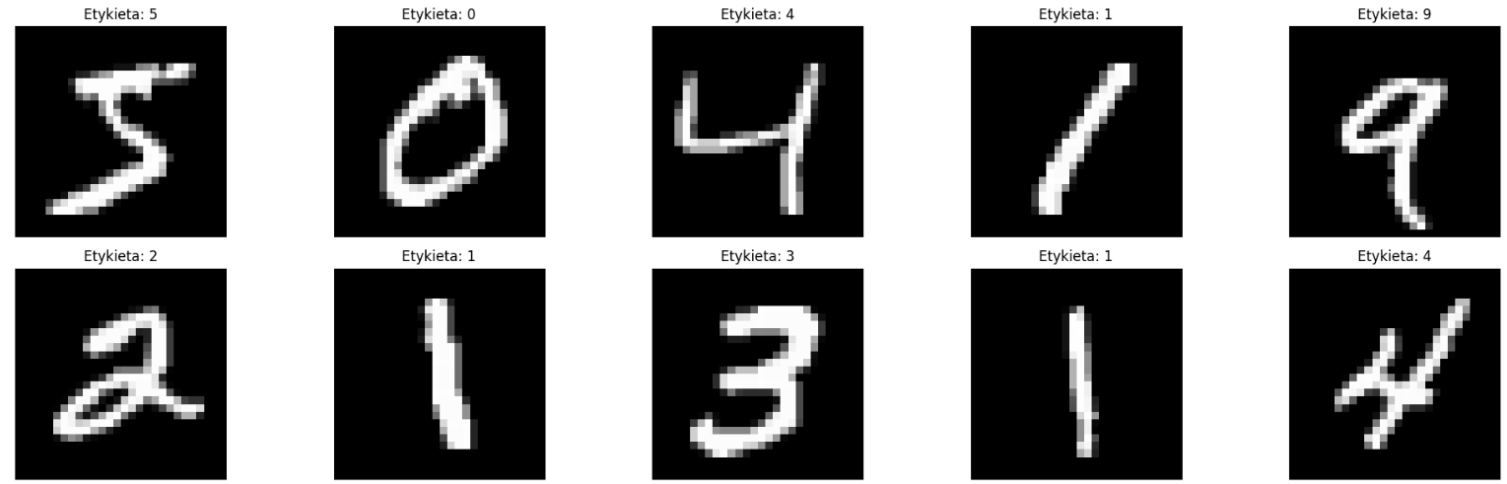
2. Wczytaj dane (`datasets.mnist.load_data()`). Znormalizuj dane do przedziału [0, 1]. Pokaż przykładowe obrazy cyfr.

## Rozwiązanie zadania:

```
#zadanie 2
#dataset z podziałem na zbiory testowe i treningowe
(train_images, train_labels), (test_images, test_labels) =
mnist.load_data()

#normalizacja
train_images = train_images / 255.0
test_images = test_images / 255.0

#rysowanie 10 przykładowych obrazków
plt.figure(figsize=(20, 6))
for i in range(10):
    plt.subplot(2, 5, i+1)
    plt.imshow(train_images[i], cmap='gray')
    plt.title(f"Etykieta: {train_labels[i]}")
    plt.axis('off')
plt.tight_layout()
plt.show()
```

## Wynik:

```
Downloading data from https://storage.googleapis.com/tensorflow/tf-keras-datasets/mnist.npz
11490434/11490434 ━━━━━━━━━━━━━━━━━━━━ 0s 0us/step
```



## Wniosek:

W tym zadaniu dane zostały zaczytane z podanego datasetu z podziałem na dane testowe i treningowe , następnie znormalizowane z postaci bitowej (z przedziału [0,255] na [0,1]), a na koniec wyświetlone przykładowe 10 cyfr wraz z ich etykietami. Zauważalny jest różny układ tych samych cyfr.

## Zadanie 3

## Treść:

3. Zbuduj model sieci CNN z jedną warstwą konwolucyjną. Skompiluj model. Przeprowadź uczenie modelu. Eksperymentalnie dobierz hiperparametry zapewniające najmniejszy błąd klasyfikacji (estymowany na zbiorze walidacyjnym). Przedstaw wyniki kilku wariantów.

## Rozwiązanie zadania:

```
#Zadanie 3

#param list
params_list = [
    {'filters': 16, 'dense_units': 32},
    {'filters': 32, 'dense_units': 64},
    {'filters': 64, 'dense_units': 128}
]

#przygotowanie danych
(train_images, train_labels), (test_images, test_labels) =
tf.keras.datasets.mnist.load_data()
train_images = train_images.reshape(-1, 28, 28, 1) / 255.0

#testowanie para listy
for params in params_list:
    print(f"\nTestowane parametry: {params}")

#budowa modelu
    model = tf.keras.Sequential([
        tf.keras.layers.Conv2D(params['filters'], (3, 3),
activation='relu', input_shape=(28, 28, 1)),
        tf.keras.layers.MaxPooling2D((2, 2)),
        tf.keras.layers.Flatten(),
        tf.keras.layers.Dense(params['dense_units'],
activation='relu'),
        tf.keras.layers.Dense(10, activation='softmax')
```

```
    ])

    model.compile(optimizer='adam',loss='sparse_categorical_crossentropy',metrics=['accuracy'])
#trening
    history = model.fit(train_images, train_labels, epochs=5,batch_size=300, validation_split=0.2, verbose=1)
#wyniki końcowe walidacji
    val_acc = history.history['val_accuracy'][-1]
    print(f"Końcowa dokładność walidacji: {val_acc:.4f}")
```

**Wynik:**

```
Testowane parametry: {'filters': 16, 'dense_units': 32}
/usr/local/lib/python3.11/dist-packages/keras/src/layers/convolutional/base_conv.py:107: UserWarning: Do not pass an `in
  super().__init__(activity_regularizer=activity_regularizer, **kwargs)
Epoch 1/5
160/160 ━━━━━━━━━━━━━━━━━━━━ 9s 51ms/step - accuracy: 0.7417 - loss: 0.9948 - val_accuracy: 0.9356 - val_loss: 0.2307
Epoch 2/5
160/160 ━━━━━━━━━━━━━━━━━━━━ 11s 59ms/step - accuracy: 0.9383 - loss: 0.2089 - val_accuracy: 0.9559 - val_loss: 0.1580
Epoch 3/5
160/160 ━━━━━━━━━━━━━━━━━━━━ 8s 47ms/step - accuracy: 0.9588 - loss: 0.1425 - val_accuracy: 0.9654 - val_loss: 0.1214
Epoch 4/5
160/160 ━━━━━━━━━━━━━━━━━━━━ 8s 48ms/step - accuracy: 0.9694 - loss: 0.1085 - val_accuracy: 0.9698 - val_loss: 0.1039
Epoch 5/5
160/160 ━━━━━━━━━━━━━━━━━━━━ 10s 47ms/step - accuracy: 0.9755 - loss: 0.0891 - val_accuracy: 0.9739 - val_loss: 0.0927
Końcowa dokładność walidacji: 0.9739

Testowane parametry: {'filters': 32, 'dense_units': 64}
Epoch 1/5
160/160 ━━━━━━━━━━━━━━━━━━━━ 12s 73ms/step - accuracy: 0.8094 - loss: 0.7284 - val_accuracy: 0.9517 - val_loss: 0.1734
Epoch 2/5
160/160 ━━━━━━━━━━━━━━━━━━━━ 22s 82ms/step - accuracy: 0.9537 - loss: 0.1590 - val_accuracy: 0.9693 - val_loss: 0.1116
Epoch 3/5
160/160 ━━━━━━━━━━━━━━━━━━━━ 19s 72ms/step - accuracy: 0.9716 - loss: 0.0996 - val_accuracy: 0.9741 - val_loss: 0.0913
Epoch 4/5
160/160 ━━━━━━━━━━━━━━━━━━━━ 21s 73ms/step - accuracy: 0.9797 - loss: 0.0741 - val_accuracy: 0.9771 - val_loss: 0.0794
Epoch 5/5
160/160 ━━━━━━━━━━━━━━━━━━━━ 11s 69ms/step - accuracy: 0.9840 - loss: 0.0587 - val_accuracy: 0.9804 - val_loss: 0.0690
Końcowa dokładność walidacji: 0.9804

Testowane parametry: {'filters': 64, 'dense_units': 128}
Epoch 1/5
160/160 ━━━━━━━━━━━━━━━━━━━━ 26s 160ms/step - accuracy: 0.8396 - loss: 0.5886 - val_accuracy: 0.9673 - val_loss: 0.1176
Epoch 2/5
160/160 ━━━━━━━━━━━━━━━━━━━━ 26s 161ms/step - accuracy: 0.9686 - loss: 0.1049 - val_accuracy: 0.9776 - val_loss: 0.0795
Epoch 3/5
160/160 ━━━━━━━━━━━━━━━━━━━━ 40s 157ms/step - accuracy: 0.9808 - loss: 0.0663 - val_accuracy: 0.9810 - val_loss: 0.0661
Epoch 4/5
160/160 ━━━━━━━━━━━━━━━━━━━━ 40s 150ms/step - accuracy: 0.9863 - loss: 0.0481 - val_accuracy: 0.9834 - val_loss: 0.0580
Epoch 5/5
160/160 ━━━━━━━━━━━━━━━━━━━━ 41s 152ms/step - accuracy: 0.9894 - loss: 0.0364 - val_accuracy: 0.9843 - val_loss: 0.0573
Końcowa dokładność walidacji: 0.9843
```

## Wniosek:

Zastosowany tu został model sieci neuronowej o 1 warstwie dla trzech przygotowanych zestawów w params_combinations. Wybrany został ten o największym accurancy, czyli model 3: {'filters': 64, 'dense_units': 128}.

## Zadanie 4

## Treść:

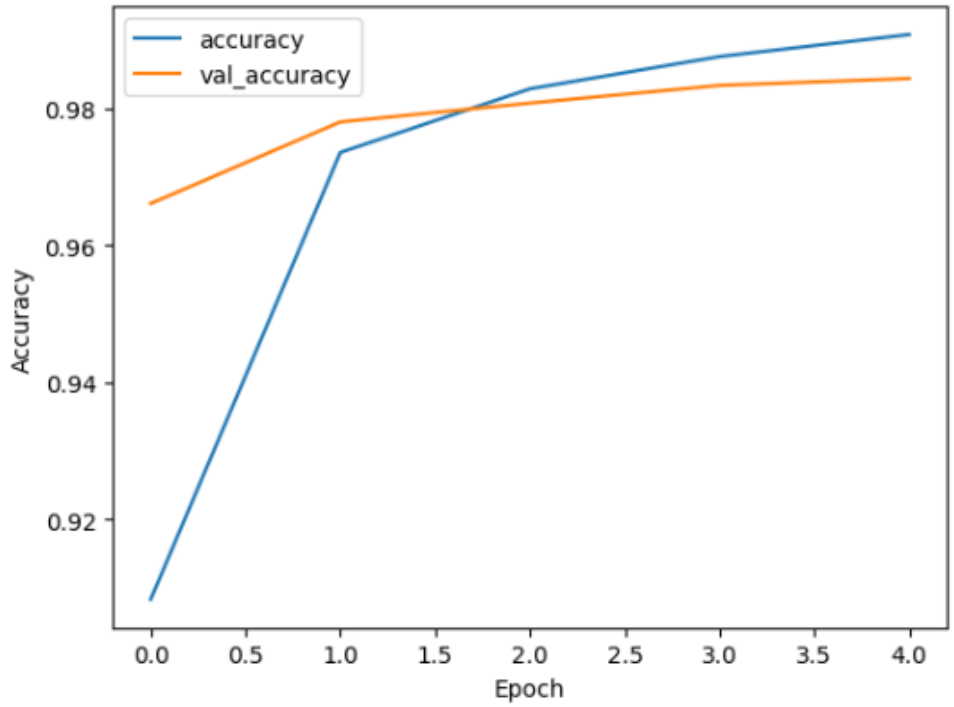
4. Dla najlepszego modelu z p. 3 pokaż krzywe zbieżności oraz *confusion matrix* dla zbioru testowego.

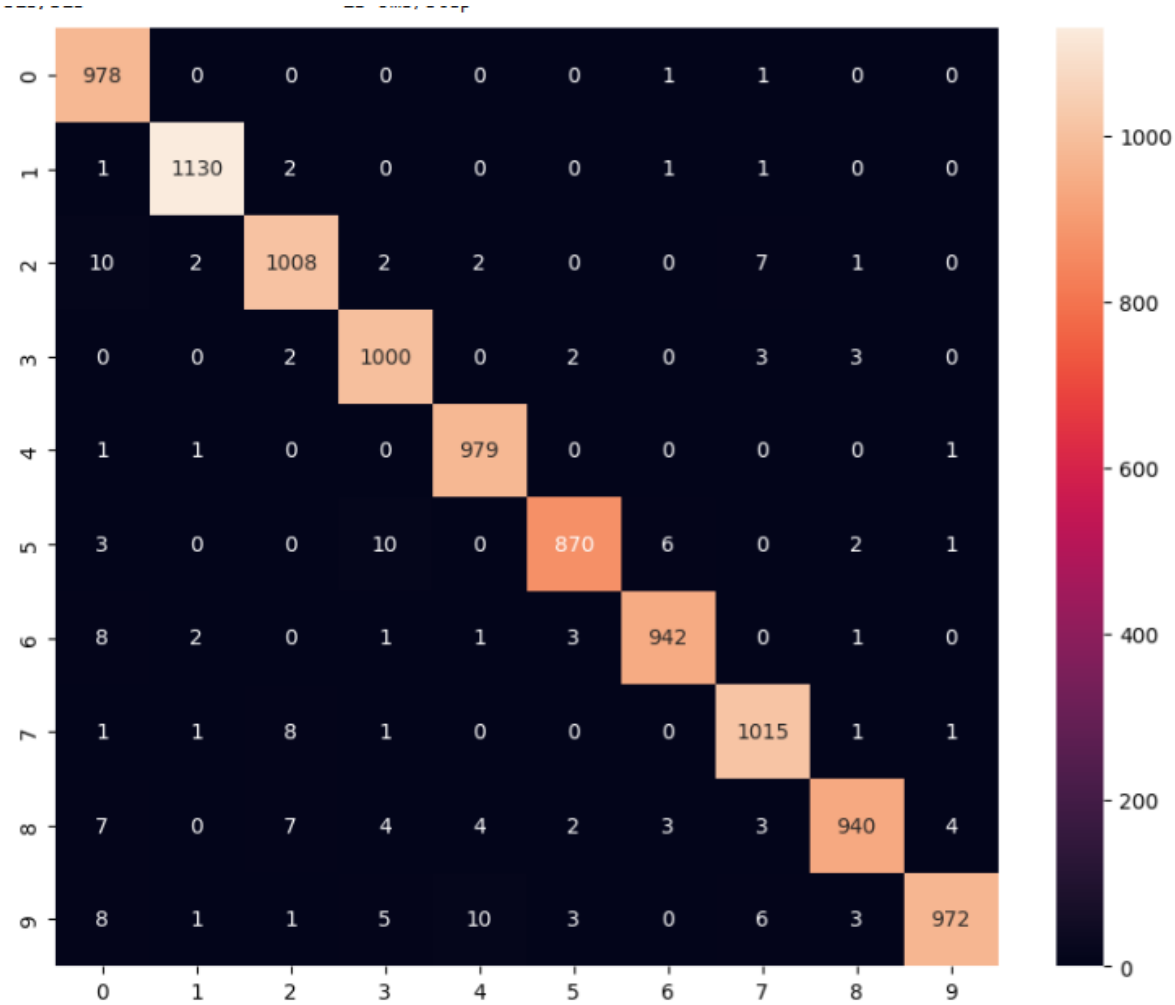
   Wskazówki:
   odpowiedzią modelu (`predictions = model.predict(test_images)`) jest macierz *N*x10, gdzie *N* jest liczbą obrazów testowych. Macierz zawiera odpowiedzi 10 neuronów wyjściowych dla każdego obrazu. Maksymalna odpowiedź dla danego obrazu wskazuje jego klasę (`pred_labels = predictions.argmax(axis=1)`).

## Rozwiązanie zadania:

```
#Zadanie 4

test_images = test_images.reshape(-1, 28, 28, 1)

plt.plot(history.history['accuracy'], label='accuracy')
plt.plot(history.history['val_accuracy'], label='val_accuracy')
plt.xlabel('Epoch')
plt.ylabel('Accuracy')
plt.legend()
plt.show()

predictions = model.predict(test_images)
pred_labels = np.argmax(predictions, axis=1)

cm = confusion_matrix(test_labels, pred_labels)
plt.figure(figsize=(10, 8))
sns.heatmap(cm, annot=True, fmt='d')
plt.show()
```

**Wynik:**


accuracy
val_accuracy
Accuracy
0.98
0.96
0.94
0.92
0.0
0.5
1.0
1.5
2.0
2.5
3.0
3.5
4.0
Epoch


0 978 0 0 0 0 0 1 1 0 0
1 1 1130 2 0 0 0 1 1 0 0
2 10 2 1008 2 2 0 0 7 1 0
3 0 0 2 1000 0 2 0 3 3 0
4 1 1 0 0 979 0 0 0 0 1
5 3 0 0 10 0 870 6 0 2 1
6 8 2 0 1 1 3 942 0 1 0
7 1 1 8 1 0 0 0 1015 1 1
8 7 0 7 4 4 2 3 3 940 4
9 8 1 1 5 10 3 0 6 3 972
0 1 2 3 4 5 6 7 8 9
1000
800
600
400
200
0

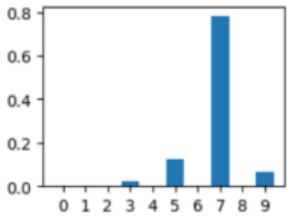
## Wniosek:

Sieć konwolucyjna, która została wybrana jako najlepsza z zadania 3 w sposób pokazany na macierzy przewidywała etykiety liczb. Niemal wszystkie liczby (powyżej 98%) zostały zaklasyfikowane poprawnie, zatem model na podstawie danych treningowych poprawnie klasyfikuje etykiety do cyfr ze zbioru testowego.

## Zadanie 5

## Treść:

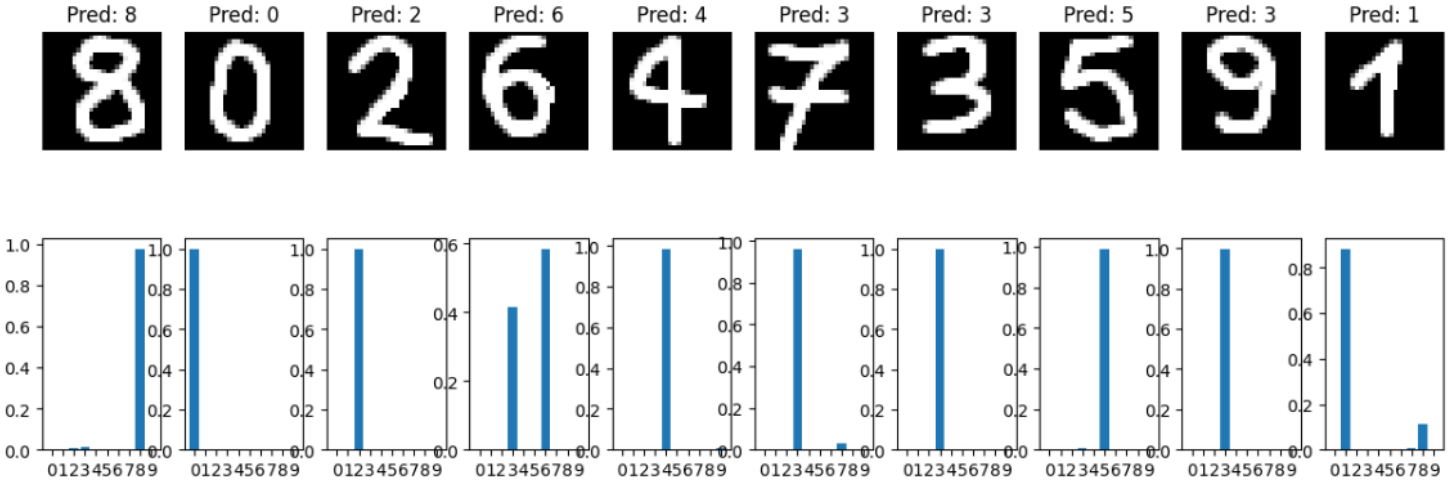
5. Wyznacz klasy przygotowanych wcześniej próbek własnego pisma. Pokaż obrazki cyfr oraz odpowiedzi sieci w postaci wykresów słupkowych postaci:




## Rozwiązanie zadania:

```
#Zadanie 5

digit_files = [f for f in os.listdir() if f.endswith('.png')]

plt.figure(figsize=(15, 5))
for i, file in enumerate(digit_files):
    img = cv2.imread(file, 0)
    img = cv2.resize(img, (28, 28))
    img = 1-img.reshape(1, 28, 28, 1) / 255.0

    pred = model.predict(img)
    pred_digit = np.argmax(pred)

    plt.subplot(2, len(digit_files), i+1)
    plt.imshow(img[0, :, :, 0], cmap='gray')
    plt.title(f"Pred: {pred_digit}")
    plt.axis('off')

    plt.subplot(2, len(digit_files), len(digit_files)+i+1)
    plt.bar(range(10), pred[0])
    plt.xticks(range(10))
plt.show()
```

## Wynik:




## Wniosek:

Wyuczona sieć konwolucyjna po otrzymaniu przykładowych plików z cyframi wyświetliła je, a następnie narysowała wykresy predykcji. Zamienione zostały kolory, stąd 1 – img.reshape, ponieważ przykładowe cyfry w porównaniu do tych z bazy są odwrócone kolorami. Poprawnie rozpoznane zostały cyfry: 8,0,2,6,4,5,1, natomiast cyfry: 7,9 nie. Jest to niestety dosyć prawdopodobne z między innymi odstającym wyglądem narysowanych cyfr od tych, na których model był uczony. Dodatkowo, cyfra 3 i 9 są całkiem podobne w wyglądzie, co również może sugerować, że ich podobieństwo ma wpływ na wynik.

## Wnioski końcowe:

Wyuczona sieć konwolucyjna po trenowaniu dokonała predykcji zaczytanych obrazów liczb. Model osiągnął wysoką dokładność ( ponad 98%) na zbiorze testowym MNIST.
Macierz pomyłek wykazała, że większość błędów dotyczyła podobnych cyfr wizualnie. Prostsze modele były nieco gorsze, a sam wynik najlepszego modelu nie jest idealny, gdyby przetestować dużo więcej kombinacji hiper parametrów, to wynik mógłby być dużo bardziej obiecujący. Trzeba jednak pamiętać, że jest to wynik na zbiorze testowym, czyli wynik poprawnych klasyfikacji danych z datasetu, a nie tych, które były wprowadzane w zadaniu 5. Model błędnie sklasyfikował niektóre ręcznie napisane cyfry np. 7, 9, co może wynikać z: między innymi różnic w stylu pisma, bądź też po prostu wpadnięcia do tych 5% źle zaklasyfikowanych liczb (jest to raczej ten rzadszy przypadek). CNN jest bardzo wrażliwe na dane wejściowe, jeżeli wyuczył się on na cyfrach, które były robione w sposób inny niż te, na których na być testowany, to może dawać złe wyniki.